Week 13: MongoDB aggregation and data modelling

Learning outcomes: CLO5

Lesson

- A **pipeline** runs stages in order:
- `$match` filter (WHERE), `$group` + `$sum` (GROUP BY), `$sort` / `$limit` (ORDER BY / LIMIT), `$project` choose fields;
- `$lookup` join into an array, `$unwind` one document per array item.
- Put `$match` early so later stages do less work.
- **Embedding:** related data inside one document; fast reads, repeated data, can grow large.
- **Referencing:** store a code and look it up, like a foreign key. Choose by how data is read and updated.

Worked example

Run `example.js` (how (`setup/README.md`)), then answer in `work.js`.

Collections for this week

Same rows and field names as MySQL, one document per row, plus `_id`. Read `cambodia`, write `student_work`.

Collection	Field	Type	Meaning
`province`	`province_pcode`	string	Province code
`province`	`province_name`	string	Province name (English)
`province`	`area_km2`	number	Area in km², one decimal place
`district`	`district_pcode`	string	District code
`district`	`district_name`	string	District name (English)
`district`	`area_km2`	number	Area in km², one decimal place
`district`	`province_pcode`	string	Province code

Links: `district.province_pcode` → `province`. MongoDB does not enforce them.

Pipeline fields: group on `$province_pcode`. `$lookup` puts matches in `districts`; after `$unwind` use `districts.district_pcode`. `district_count` is computed, not stored.

More: data guide (`data/README.md`).

Lab tasks

1. Count districts per `province_pcode`, sorted by code. Translate a SQL `GROUP BY … ORDER BY … LIMIT` into `$group`, `$sort`, `$limit` and compare.
2. Show each district with its province name (`$lookup`).
3. Attach districts to provinces, `$unwind`, and compare the count with the SQL join.
4. Remove `_id` with `$project`; compare values with SQL.
5. Design an embedded province-with-districts model from the existing data. How do you keep it updated? How does it grow?

Homework

1. Write three pipelines using $match, $group, $project, $lookup and $unwind.
2. For two project queries: embed or reference? Why?

Capstone sprint

C13: Three pipelines with output, and your document-model choice.

Submit

Answers in `work.js`, reflection in `reflection.md`, output in `results/`. Commit and push. See SUBMISSIONS.md.